

etl_imdb

June 21, 2026

1 ETL para el Dataset de IMDb

Este cuaderno contiene el flujo de Extracción, Transformación y Carga de los datos oficiales de IMDb, descargados de <https://developer.imdb.com/non-commercial-datasets/>. Este proceso consiste en procesar los ficheros en formato TSV, limpiarlos, aplicar filtros y cargarlos en un modelo dimensional en Aiven MySQL Cloud.

1.1 Importación de librerías y configuración

Comenzamos importando las dependencias necesarias. Usamos **pandas** para la manipulación de datos, **sqlalchemy** para la carga en la base de datos y **ast** para desanidar columnas más complejas.

```
[21]: import os
import pandas as pd
import ast
from sqlalchemy import create_engine
from dotenv import load_dotenv, find_dotenv

os.makedirs('../data', exist_ok=True)
```

1.2 Extracción y transformación de tablas

1.2.1 H_RATINGS (Tabla de Hechos de Valoraciones)

Esta es una de nuestras tablas de hechos, que almacena las valoraciones de las películas. Para evitar el sesgo con películas desconocidas y optimizar el almacenamiento en el cluster de Aiven, aplicamos un filtro para conservar solo los títulos con mínimo 50 votos.

```
[22]: # Lectura del archivo original
H_RATINGS = pd.read_csv('../tsv/title.ratings.tsv', sep='\t', na_values='\\N',
    ↪index_col=0, header=0)
H_RATINGS.index.names = ['titleID']

# Filtro por 50 votos minimos para quitar las peliculas desconocidas
H_RATINGS = H_RATINGS[H_RATINGS['numVotes'] >= 50]

print(f"Número de títulos con valoraciones filtradas: {len(H_RATINGS)}")
print(H_RATINGS.head())
```

Número de títulos con valoraciones filtradas: 594332

	averageRating	numVotes
titleID		
tt0000001	5.7	2189
tt0000002	5.5	307
tt0000003	6.5	2291
tt0000004	5.1	195
tt0000005	6.2	3023

```
[23]: # Exportamos a la carpeta
H_RATINGS.to_csv('data/H_RATINGS.csv')
print('H_RATINGS guardado.\n')
```

H_RATINGS guardado.

1.2.2 DIM_TITLES (Dimensión de Títulos)

La dimensión de títulos es una unión de la información básica del título (`title.basics.tsv`) y los datos de los episodios (`title.episode.tsv`). Solo procesamos los títulos que pasaron el filtro anterior para optimizar el rendimiento.

```
[24]: # Carga de metadatos básicos
title_basics = pd.read_csv('../tsv/title.basics.tsv', sep='\t',
    ↳na_values='\\N', low_memory=False)
print(title_basics.head())
```

	tconst	titleType	primaryTitle	originalTitle	\
0	tt0000001	short	Carmencita	Carmencita	
1	tt0000002	short	Le clown et ses chiens	Le clown et ses chiens	
2	tt0000003	short	Poor Pierrot	Pauvre Pierrot	
3	tt0000004	short	Un bon bock	Un bon bock	
4	tt0000005	short	Blacksmith Scene	Blacksmith Scene	

	isAdult	startYear	endYear	runtimeMinutes	genres
0	0	1894.0	NaN	1	Documentary,Short
1	0	1892.0	NaN	5	Animation,Short
2	0	1892.0	NaN	5	Animation,Comedy,Romance
3	0	1892.0	NaN	12	Animation,Short
4	0	1893.0	NaN	1	Short

```
[25]: # Carga de episodios
title_episode = pd.read_csv('../tsv/title.episode.tsv', sep='\t',
    ↳na_values='\\N', low_memory=False)
print(title_episode.head())
```

	tconst	parentTconst	seasonNumber	episodeNumber
0	tt0031458	tt32857063	NaN	NaN
1	tt0041951	tt0041038	1.0	9.0

2	tt0042816	tt0989125	1.0	17.0
3	tt0042889	tt0989125	NaN	NaN
4	tt0043426	tt0040051	3.0	42.0

```
[26]: # Merging y limpieza inicial de columnas
DIM_TITLES = pd.merge(title_basics, title_episode, on='tconst', how='left')
DIM_TITLES = DIM_TITLES.rename(columns={'tconst': 'titleID', 'parentTconst': 'parentTitleID'})
DIM_TITLES = DIM_TITLES.drop(columns='originalTitle')

# Filtro para alinear con la tabla de hechos
DIM_TITLES = DIM_TITLES[DIM_TITLES['titleID'].isin(H_RATINGS.index)]

# Conversión de tipos de datos
DIM_TITLES['startYear'] = pd.to_numeric(DIM_TITLES['startYear'], errors='coerce').astype('Int64')
DIM_TITLES['endYear'] = pd.to_numeric(DIM_TITLES['endYear'], errors='coerce').astype('Int64')
DIM_TITLES['seasonNumber'] = pd.to_numeric(DIM_TITLES['seasonNumber'], errors='coerce').astype('Int64')
DIM_TITLES['episodeNumber'] = pd.to_numeric(DIM_TITLES['episodeNumber'], errors='coerce').astype('Int64')
DIM_TITLES['runtimeMinutes'] = pd.to_numeric(DIM_TITLES['runtimeMinutes'], errors='coerce').astype('Int64')

# Reemplazo valores incoherentes (> 100 temporadas) por nulos (pd.NA)
DIM_TITLES.loc[DIM_TITLES['seasonNumber'] > 100, 'seasonNumber'] = pd.NA

# Reemplazo producciones extremadamente largas (más de 8 horas, 480min) por pd.NA
DIM_TITLES.loc[DIM_TITLES['runtimeMinutes'] > 480, 'runtimeMinutes'] = pd.NA
DIM_TITLES.loc[DIM_TITLES['runtimeMinutes'] <= 0, 'runtimeMinutes'] = pd.NA

# Reordenar columnas antes de procesar fechas y géneros
column_order = [
    'titleID', 'parentTitleID', 'titleType', 'primaryTitle',
    'isAdult', 'startYear', 'endYear', 'seasonNumber',
    'episodeNumber', 'runtimeMinutes', 'genres'
]
DIM_TITLES = DIM_TITLES[column_order]

# Conjunto de códigos para filtros posteriores
titulos_validos = set(DIM_TITLES['titleID'])

print(f"Dimension DIM_TITLES generada con {len(DIM_TITLES)} registros.")
print(DIM_TITLES.head(3))
```

Dimension DIM_TITLES generada con 594332 registros.

	titleID	parentTitleID	titleType	primaryTitle	isAdult	\
0	tt00000001	NaN	short	Carmencita	0	
1	tt00000002	NaN	short	Le clown et ses chiens	0	
2	tt00000003	NaN	short	Poor Pierrot	0	

	startYear	endYear	seasonNumber	episodeNumber	runtimeMinutes	\
0	1894	<NA>	<NA>	<NA>	1	
1	1892	<NA>	<NA>	<NA>	5	
2	1892	<NA>	<NA>	<NA>	5	

	genres
0	Documentary,Short
1	Animation,Short
2	Animation,Comedy,Romance

1.2.3 H_PRINCIPALS (Tabla puente entre Títulos y Personas)

Como el archivo original de IMDb (`title.principals.tsv`) es demasiado grande, procesamos por bloques para no saturar la memoria RAM.

```
[27]: archivo_destino = 'data/H_PRINCIPALS.csv'
if os.path.exists(archivo_destino): os.remove(archivo_destino)

names_activos = set() # Para guardar los IDs de personas reales y filtrar
↳ DIM_NAMES después

# Leemos en bloques de 500k filas
for chunk in pd.read_csv('../tsv/title.principals.tsv', sep='\t',
↳ na_values='\\N', chunksize=500000, low_memory=False):
    chunk.rename(columns={'tconst': 'titleID', 'nconst': 'nameID'},
↳ inplace=True)

    # Filtrar por los títulos válidos
    chunk_filtrado = chunk[chunk['titleID'].isin(titulos_validos)].copy()

    # Filtrar solo categorías clave (actor, actress, director)
    roles_clave = ['actor', 'actress', 'director']
    chunk_filtrado = chunk_filtrado[chunk_filtrado['category'].
↳ isin(roles_clave)]

    if not chunk_filtrado.empty:
        # Desanidado de la columna characters
        chunk_filtrado['characters'] = chunk_filtrado['characters'].apply(
            lambda x: ast.literal_eval(x) if isinstance(x, str) and x.
↳ startswith('[') else x
        )
        # Expandir la lista a filas independientes
```

```

        chunk_filtrado = chunk_filtrado.explode('characters')

        # Guardar los personajes asociados a estos títulos
        names_activos.update(chunk_filtrado['nameID'].dropna().unique())

        # Guardar en el CSV
        escribir_cabecera = not os.path.exists(archivo_destino)
        chunk_filtrado.to_csv(archivo_destino, mode='a',
        ↪header=escribir_cabecera, index=False)

    print("H_PRINCIPALS filtrado y guardado")

```

H_PRINCIPALS filtrado y guardado

1.2.4 DIM_NAMES (Dimensión de Personas)

En esta tabla almacenamos los datos de actores, guionistas y otros profesionales. Al igual que la tabla anterior, la leemos por bloques y la filtramos con la lista de IDs activos (names_activos) de la celda anterior.

```

[28]: archivo_names_destino = 'data/DIM_NAMES.csv'
    if os.path.exists(archivo_names_destino): os.remove(archivo_names_destino)

    for chunk in pd.read_csv('../tsv/name.basics.tsv', sep='\t', na_values='\\N',
    ↪chunksize=500000, low_memory=False):
        chunk.index.names = ['nameID']
        if 'nconst' in chunk.columns:
            chunk.rename(columns={'nconst': 'nameID'}, inplace=True)

        # Filtrar: solo personas que aparecen en nuestras películas filtradas
        chunk_filtrado = chunk[chunk['nameID'].isin(names_activos)].copy()

        if not chunk_filtrado.empty:
            # limpieza y transformaciones
            chunk_filtrado = chunk_filtrado.drop(columns='knownForTitles',
            ↪errors='ignore')
            chunk_filtrado['primaryProfession'] =
            ↪chunk_filtrado['primaryProfession'].str.split(',')
            chunk_filtrado = chunk_filtrado.explode('primaryProfession')
            chunk_filtrado['birthYear'] = pd.
            ↪to_numeric(chunk_filtrado['birthYear'], errors='coerce').astype('Int64')
            chunk_filtrado['deathYear'] = pd.
            ↪to_numeric(chunk_filtrado['deathYear'], errors='coerce').astype('Int64')

            # Guardar
            escribir_cabecera = not os.path.exists(archivo_names_destino)

```

```

        chunk_filtrado.to_csv(archivo_names_destino, mode='a',
↪header=escribir_cabecera, index=False)

print("DIM_NAMES guardado")

```

DIM_NAMES guardado

1.2.5 DIM_FECHA

Para ayudarnos posteriormente con el análisis temporal, desacoplamos la columna `startYear` en una dimensión independiente.

```

[29]: # Obtener años de lanzamiento únicos de los títulos válidos
anyos = DIM_TITLES['startYear'].dropna().unique()
DIM_FECHA = pd.DataFrame(anyos, columns=['startYear']).sort_values('startYear')
DIM_FECHA = DIM_FECHA.reset_index(drop=True)

# Generar ID como clave primaria
DIM_FECHA['fechaID'] = DIM_FECHA.index + 1

# Mapear fechaID a DIM_TITLES
DIM_TITLES = DIM_TITLES.merge(DIM_FECHA, on='startYear', how='left')
DIM_TITLES = DIM_TITLES.drop(columns=['startYear'])
DIM_TITLES = DIM_TITLES.rename(columns={'fechaID': 'startYear'})
DIM_TITLES['startYear'] = DIM_TITLES['startYear'].astype('Int64')

# Renombrar columnas finales
DIM_FECHA = DIM_FECHA.rename(columns={'fechaID': 'FechaID', 'startYear': 'Año'})

print(DIM_FECHA.head())

```

	Año	FechaID
0	1874	1
1	1878	2
2	1881	3
3	1882	4
4	1883	5

1.2.6 DIM_GENRES

Los títulos tienen múltiples géneros separados por comas. Para evitar almacenar datos con más de un valor, descomponemos los géneros en una tabla de dimensiones (DIM_GENRES) y creamos la tabla puente de hechos H_TITLES_GENRES.

```

[30]: # Normalización de géneros únicos
genres_df = DIM_TITLES[['genres']].dropna().copy()
genres_df['genres'] = genres_df['genres'].str.split(',')
genres_df = genres_df.explode('genres')

```

```

DIM_GENRES = pd.DataFrame(genres_df['genres'].unique(), columns=['genreName']).
    ↪sort_values('genreName')
DIM_GENRES = DIM_GENRES.reset_index(drop=True)
DIM_GENRES['genreID'] = DIM_GENRES.index + 1

# Generación de la tabla puente de hechos muchos a muchos (M:N)
H_TITLES_GENRES = DIM_TITLES[['titleID', 'genres']].dropna().copy()
H_TITLES_GENRES['genres'] = H_TITLES_GENRES['genres'].str.split(',')
H_TITLES_GENRES = H_TITLES_GENRES.explode('genres')

H_TITLES_GENRES = H_TITLES_GENRES.merge(DIM_GENRES, left_on='genres',
    ↪right_on='genreName')
H_TITLES_GENRES = H_TITLES_GENRES[['titleID', 'genreID']]

# Eliminar columna géneros de DIM_TITLES (ya está en la tabla intermedia)
DIM_TITLES = DIM_TITLES.drop(columns=['genres'])

```

1.2.7 Exportación de las tablas a archivos CSV

Guardamos los conjuntos de datos limpios como ficheros CSV para cargarlos después en la base de datos.

```

[31]: # Guardado de las tablas restantes
DIM_TITLES.to_csv('data/DIM_TITLES.csv', index=False)
DIM_FECHA.to_csv('data/DIM_FECHA.csv', index=False)
DIM_GENRES.to_csv('data/DIM_GENRES.csv', index=False)
H_TITLES_GENRES.to_csv('data/H_TITLES_GENRES.csv', index=False)
print('Tablas de títulos, fechas y géneros guardadas.')

```

Tablas de títulos, fechas y géneros guardadas.

1.3 Carga de datos en Aiven MySQL Cloud

1.3.1 Configuración de la conexión

En esta sección definimos la conexión en la instancia de MySQL en Aiven Cloud.

```

[32]: import sys

# Configuración de las credenciales de Aiven
# Cargamos las variables de entorno
load_dotenv(find_dotenv())

# Leemos las variables
USER = os.getenv('DB_USER', 'avnadmin')
PASSWORD = os.getenv('DB_PASSWORD')
HOST = os.getenv('DB_HOST')
PORT = os.getenv('DB_PORT', '10854')
DATABASE = os.getenv('DB_DATABASE', 'imdb_tfm')

```

```

# Ruta al certificado SSL
ssl_ca_path = os.path.abspath('../ca.pem')

if not os.path.exists(ssl_ca_path):
    print(f"El archivo de certificado CA '{ssl_ca_path}' no se encuentra en el_
↳ directorio actual.", file=sys.stderr)
if not PASSWORD:
    print("ERROR: No se ha podido cargar la contraseña desde el archivo .env._
↳ Revisa la ruta del archivo.", file=sys.stderr)

# URL de conexión para MySQL
connection_string = f"mysql+pymysql://{USER}:{PASSWORD}@{HOST}:{PORT}/
↳ {DATABASE}"

engine = create_engine(
    connection_string,
    connect_args={"ssl": {"ca": ssl_ca_path}}
)

```

1.3.2 Carga de datos

```

[33]: TABLA = 'DIM_FECHA'
      CSV = 'data/DIM_FECHA.csv'

print(f"Vacando {TABLA}...")
with engine.connect() as conn:
    dbapi_conn = conn.dbapi_connection if hasattr(conn, 'dbapi_connection')_
↳ else conn.connection
    with dbapi_conn.cursor() as cursor:
        cursor.execute("SET FOREIGN_KEY_CHECKS = 0;")
        cursor.execute(f"TRUNCATE TABLE {TABLA};")
        cursor.execute("SET FOREIGN_KEY_CHECKS = 1;")
    dbapi_conn.commit()

print(f"Procesando {TABLA}...")
df = pd.read_csv(CSV)
df = df.drop_duplicates(subset=['FechaID'])

print(f"Cargando {len(df)} filas...")
with engine.connect() as conn:
    trans = conn.begin()
    try:
        df.to_sql(name=TABLA, con=conn, if_exists='append', index=False,_
↳ chunksize=15000)
        trans.commit()
        print(f"{TABLA} cargada")

```



```

except Exception as e:
    trans.rollback()
    print(f"Error en {TABLA}: {e}")

```

Vaciando DIM_FECHA...
 Procesando DIM_FECHA...
 Cargando 146 filas...
 DIM_FECHA cargada

```

[34]: TABLA = 'DIM_GENRES'
      CSV = 'data/DIM_GENRES.csv'

      print(f"Vaciando {TABLA}...")
      with engine.connect() as conn:
          dbapi_conn = conn.dbapi_connection if hasattr(conn, 'dbapi_connection')
          else conn.connection
          with dbapi_conn.cursor() as cursor:
              cursor.execute("SET FOREIGN_KEY_CHECKS = 0;")
              cursor.execute(f"TRUNCATE TABLE {TABLA};")
              cursor.execute("SET FOREIGN_KEY_CHECKS = 1;")
          dbapi_conn.commit()

      print(f"Procesando {TABLA}...")
      df = pd.read_csv(CSV)
      df['genreName'] = df['genreName'].fillna('Unknown Genre')
      df = df.drop_duplicates(subset=['genreID'])

      print(f"Cargando {len(df)} filas...")
      with engine.connect() as conn:
          trans = conn.begin()
          try:
              df.to_sql(name=TABLA, con=conn, if_exists='append', index=False,
              chunksize=15000)
              trans.commit()
              print(f"{TABLA} cargada")
          except Exception as e:
              trans.rollback()
              print(f"Error en {TABLA}: {e}")

```

Vaciando DIM_GENRES...
 Procesando DIM_GENRES...
 Cargando 28 filas...
 DIM_GENRES cargada

```

[35]: TABLA = 'DIM_TITLES'
      CSV = 'data/DIM_TITLES.csv'

      print(f"Vaciando {TABLA}...")

```

```

with engine.connect() as conn:
    dbapi_conn = conn.dbapi_connection if hasattr(conn, 'dbapi_connection')
    else conn.connection
    with dbapi_conn.cursor() as cursor:
        cursor.execute("SET FOREIGN_KEY_CHECKS = 0;")
        cursor.execute(f"TRUNCATE TABLE {TABLA};")
        cursor.execute("SET FOREIGN_KEY_CHECKS = 1;")
    dbapi_conn.commit()

print(f"Procesando {TABLA}...")
df = pd.read_csv(CSV)
df['primaryTitle'] = df['primaryTitle'].fillna('Untitled / Unknown')
df['titleType'] = df['titleType'].fillna('unknown')
df = df.drop_duplicates(subset=['titleID'])

print(f"Cargando {len(df)} filas...")
with engine.connect() as conn:
    trans = conn.begin()
    try:
        df.to_sql(name=TABLA, con=conn, if_exists='append', index=False,
        chunksize=15000)
        trans.commit()
        print(f"{TABLA} cargada")
    except Exception as e:
        trans.rollback()
        print(f"Error en {TABLA}: {e}")

```

Vaciando DIM_TITLES...
 Procesando DIM_TITLES...
 Cargando 594332 filas...
 DIM_TITLES cargada

```

[36]: TABLA = 'DIM_NAMES'
      CSV = 'data/DIM_NAMES.csv'

print(f"Vaciando {TABLA}...")
with engine.connect() as conn:
    dbapi_conn = conn.dbapi_connection if hasattr(conn, 'dbapi_connection')
    else conn.connection
    with dbapi_conn.cursor() as cursor:
        cursor.execute("SET FOREIGN_KEY_CHECKS = 0;")
        cursor.execute(f"TRUNCATE TABLE {TABLA};")
        cursor.execute("SET FOREIGN_KEY_CHECKS = 1;")
    dbapi_conn.commit()

print(f"Procesando {TABLA}...")
df = pd.read_csv(CSV)

```

```

df['primaryName'] = df['primaryName'].fillna('Unknown Name')
df['primaryProfession'] = df['primaryProfession'].fillna('unknown')
df = df.drop_duplicates(subset=['nameID', 'primaryProfession'])

print(f"Cargando {len(df)} filas...")
with engine.connect() as conn:
    trans = conn.begin()
    try:
        df.to_sql(name=TABLA, con=conn, if_exists='append', index=False,
        ↪chunksize=15000)
        trans.commit()
        print(f"{TABLA} cargada")
    except Exception as e:
        trans.rollback()
        print(f"Error en {TABLA}: {e}")

```

Vaciando DIM_NAMES...

Procesando DIM_NAMES...

Cargando 1540273 filas...

DIM_NAMES cargada

```

[37]: TABLA = 'H_RATINGS'
      CSV = 'data/H_RATINGS.csv'

print(f"Vaciando {TABLA}...")
with engine.connect() as conn:
    dbapi_conn = conn.dbapi_connection if hasattr(conn, 'dbapi_connection')
    ↪else conn.connection
    with dbapi_conn.cursor() as cursor:
        cursor.execute("SET FOREIGN_KEY_CHECKS = 0;")
        cursor.execute(f"TRUNCATE TABLE {TABLA};")
        cursor.execute("SET FOREIGN_KEY_CHECKS = 1;")
    dbapi_conn.commit()

print(f"Procesando {TABLA}...")
df = pd.read_csv(CSV)
df = df.drop_duplicates(subset=['titleID'])

print(f"Cargando {len(df)} filas...")
with engine.connect() as conn:
    trans = conn.begin()
    try:
        df.to_sql(name=TABLA, con=conn, if_exists='append', index=False,
        ↪chunksize=15000)
        trans.commit()
        print(f"{TABLA} cargada")
    except Exception as e:

```

```
trans.rollback()
print(f"Error en {TABLA}: {e}")
```

Vaciando H_RATINGS...
Procesando H_RATINGS...
Cargando 594332 filas...
H_RATINGS cargada

```
[38]: TABLA = 'H_TITLES_GENRES'
      CSV = 'data/H_TITLES_GENRES.csv'

      print(f"Vaciando {TABLA}...")
      with engine.connect() as conn:
          dbapi_conn = conn.dbapi_connection if hasattr(conn, 'dbapi_connection')
          ↪else conn.connection
          with dbapi_conn.cursor() as cursor:
              cursor.execute("SET FOREIGN_KEY_CHECKS = 0;")
              cursor.execute(f"TRUNCATE TABLE {TABLA};")
              cursor.execute("SET FOREIGN_KEY_CHECKS = 1;")
          dbapi_conn.commit()

      print(f"Procesando {TABLA}...")
      df = pd.read_csv(CSV)
      df = df.drop_duplicates(subset=['titleID', 'genreID'])

      print(f"Cargando {len(df)} filas...")
      with engine.connect() as conn:
          trans = conn.begin()
          try:
              df.to_sql(name=TABLA, con=conn, if_exists='append', index=False,
              ↪chunksize=15000)
              trans.commit()
              print(f"{TABLA} cargada")
          except Exception as e:
              trans.rollback()
              print(f"Error en {TABLA}: {e}")
```

Vaciando H_TITLES_GENRES...
Procesando H_TITLES_GENRES...
Cargando 1294624 filas...
H_TITLES_GENRES cargada

```
[39]: TABLA = 'H_PRINCIPALS'
      CSV = 'data/H_PRINCIPALS.csv'

      print(f"Vaciando {TABLA}...")
      with engine.connect() as conn:
```

```

    dbapi_conn = conn.dbapi_connection if hasattr(conn, 'dbapi_connection')
else conn.connection
    with dbapi_conn.cursor() as cursor:
        cursor.execute("SET FOREIGN_KEY_CHECKS = 0;")
        cursor.execute(f"TRUNCATE TABLE {TABLA};")
        cursor.execute("SET FOREIGN_KEY_CHECKS = 1;")
    dbapi_conn.commit()

print(f"Procesando {TABLA}...")
df = pd.read_csv(CSV)
df['category'] = df['category'].fillna('unknown')
df = df.drop_duplicates(subset=['titleID', 'ordering', 'nameID', 'category'])

print(f"Cargando {len(df)} filas...")
with engine.connect() as conn:
    trans = conn.begin()
    try:
        df.to_sql(name=TABLA, con=conn, if_exists='append', index=False,
chunksize=15000)
        trans.commit()
        print(f"{TABLA} cargada")
    except Exception as e:
        trans.rollback()
        print(f"Error en {TABLA}: {e}")

```

Vaciando H_PRINCIPALS...
Procesando H_PRINCIPALS...
Cargando 5654451 filas...
H_PRINCIPALS cargada